

Assignment 3

(Due: 5/6)

Please submit a **.docx** or **.pdf** file for your written responses and the requested screenshots. Please be sure to also submit all your Python code as a **.py** file. Please **do not** use zip.

1. (4 pts) CIDR Review

Please refer to the **CIDR_Notation_Guide** that was posted on D2L under Week 4 for background reading on this.

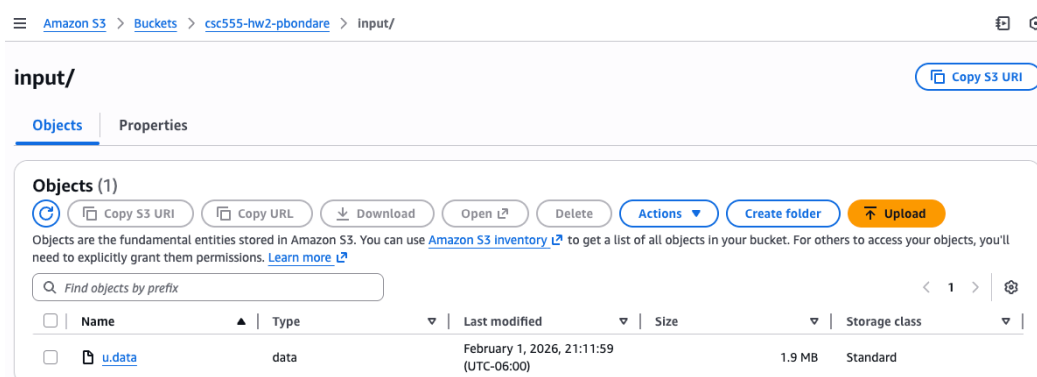
Find the number of IP addresses span by the following CIDR range: 140.192.182.72/32.

(note that this was the CIDR range that AWS filled out when I chose MyIP as I was configuring the inbound rules for the **ElasticMapReduce-master** security group to allow SSHing into the Hadoop primary node over port 22).

2. (4 pts) Copying Data: Laptop -> S3

In this exercise we will import a data file into the **csc555-hw2-<youriamusername>/input** S3 folder that we created on the last assignment.

Download the **u.data** file from D2L to your laptop and upload it to your S3 folder. Take a **screenshot** of your S3 folder with the data file in it (as below).



3. (8 pts) Elevate the Privileges of the EMR Service Role and the EC2 Instance Profile

As we have discussed in class, the AWS EMR service utilizes various other AWS services under the hood: for example, it uses the AWS EC2 service to create the cluster nodes, it needs to be able to read from AWS S3 for input data, and write to S3 to save any results, etc.

- An **EMR service role** is an IAM role that grants the EMR service permission to access and manage AWS resources like EC2 instances, S3, security groups, logs etc. It is crucial for provisioning clusters, launching instances, and performing automated tasks, functioning independently of user-specific permissions.
- An **EC2 instance profile** is a secure container for an IAM role, allowing us to pass temporary permissions to an Amazon EC2 instance when it launches, eliminating the need to embed AWS access keys directly on the instance. It acts as an identity for our EC2 instance, granting it access to other AWS services like S3 using temporary credentials provided via the instance metadata service, enhancing security and simplifying credential management.

- a) (4 pts) Elevate the privileges of the default **AmazonEMR-ServiceRole** so that it has administrator access to all AWS resources. Provide a **screenshot** of your service role's permissions (as below).

The screenshot shows the AWS IAM console interface for the role **AmazonEMR-ServiceRole-20260118T222143**. The left sidebar shows the navigation menu with 'Roles' selected. The main content area displays the role's summary and permissions. The 'Permissions policies' section shows three policies attached to the role:

Policy name	Type	Attached entities
AdministratorAccess	AWS managed - job function	5
AmazonEMR-ServiceR...	Customer managed	1
AmazonEMRService...	AWS managed	1

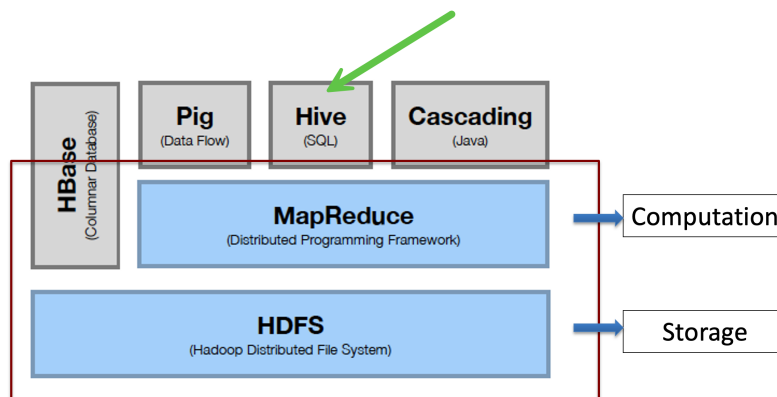
- b) (4 pts) Elevate the privileges of the default **AmazonEMR-InstanceProfile** so that it has administrator access to all AWS resources. Provide a **screenshot** of your instance profile's permissions.

The screenshot displays the AWS IAM console interface. On the left, a sidebar contains navigation links for 'Identity and Access Management (IAM)', 'Dashboard', 'Access Management' (with sub-links for User groups, Users, Roles, Policies, Identity providers, Account settings, Root access management, and Temporary delegation requests), and 'Access reports' (with sub-links for Access Analyzer, Resource analysis, Unused access, Analyzer settings, and Credential report). The main content area is titled 'AmazonEMR-InstanceProfile-20260118T222128'. It features a 'Summary' section with details such as 'Creation date' (January 18, 2026, 22:21 UTC-06:00), 'ARN' (arn:aws:iam::796466898304:role/service-role/AmazonEMR-InstanceProfile-20260118T222128), 'Instance profile ARN' (arn:aws:iam::796466898304:instance-profile/AmazonEMR-InstanceProfile-20260118T222128), 'Last activity' (6 minutes ago), and 'Maximum session duration' (1 hour). Below the summary, there are tabs for 'Permissions', 'Trust relationships', 'Tags', 'Last Accessed', and 'Revoke sessions'. The 'Permissions' tab is active, showing 'Permissions policies (2)'. It includes a search bar, a 'Filter by Type' dropdown set to 'All types', and a table listing the policies: 'AdministratorAccess' (AWS managed - job function, 5 attached entities) and 'AmazonEMR-Instance...' (Customer managed, 1 attached entity).

4. (30 pts) Creating a Hive Table and Querying Using SQL

Recall the Hadoop architecture from class:

Hadoop Stack



Hive allows us to use SQL to query the underlying data that is stored in HDFS as loose files. In this part of the homework, we are going to use Hive on the EMR cluster that we created to load a data set from S3 into HDFS and then query it using SQL.

- a) **(4 pts)** Use the EMR cluster that you created in Assignment 2 to clone a new Hadoop cluster. Please make sure to include your EC2 key pair as we will need that to SSH into the primary node.

b) (4 pts) SSH into the primary node of the new EMR cluster and provide a **screenshot** of the output of the following command: **hadoop fs -ls** (note that you have to SSH using the **hadoop** user for this command to work, as we experienced during class)

c) (6 pts) Copy over the **u.data** file that you uploaded to S3 into the local file system of the primary node using the following command:

```
aws s3 cp s3://csc555-hw2-<youriamusername>/input/u.data u.data
```

Provide a **screenshot** of the output of this command.

d) (4 pts) Verify that the file has been successfully copied over by running the following commands:

```
pwd
```

```
ls -al
```

Provide a **screenshot** of the output of these commands.

e) (6 pts) The **u.data** file contains the MovieLens data set consisting of 100K movie ratings (1-5) from 943 users on 1682 movies. The data is a tab separated list of **user id | item id | rating | timestamp**, where the time stamps are Unix seconds since 1/1/1970 UTC.

Start a Hive shell using the **hive** command, and create a table called **u_data** using the **u.data** file that we brought into the primary node, as we did in class using the code in the slide below.

Hive: Create Table

```
➤ CREATE TABLE u_data ( userid INT,  
    movieid INT,  
    rating INT,  
    unixtime STRING)  
ROW FORMAT DELIMITED FIELDS  
TERMINATED BY '\t' STORED AS TEXTFILE; (not compressed)  
  
➤ LOAD DATA LOCAL INPATH '/home/hadoop/u.data'  
OVERWRITE INTO TABLE u_data;
```

f) (6 pts) Run the following queries using Hive on the **u_data** table and provide a **screenshot** of the results for each one:

```
SELECT * FROM u_data WHERE userid = 550;
```

```
SELECT userid, AVG(rating) as AR from u_data GROUP BY userid ORDER BY AR;
```

5. (22 pts) Transforming the Data Using Python

The Unix timestamps are difficult to work with. We will transform them into weekdays using Python.

a) (6 pts) Exit the hive shell using the **exit;** command. The **weekly_mapper.py** file in D2L contains simple Python code that transforms a Unix timestamp into weekdays. Download the **weekly_mapper.py** file from D2L and bring it all the way to the primary node, following the same steps you did for the **u.data** (laptop -> S3 -> primary node).

Provide a **screenshot** of the output of your **aws s3 cp** command.

b) (4 pts) Verify that the file has been successfully copied over by running the following commands:

```
pwd
```

```
ls -al
```

Provide a screenshot of the output of these commands.

c) (6 pts) Start a hive shell again and type the following commands to create a new table called **u_data_new** with a transformed date column that includes just the weekday:

Custom Hive Transformation

```
CREATE TABLE u_data_new ( userid INT,  
movieid INT, rating INT, weekday String)  
ROW FORMAT DELIMITED FIELDS  
TERMINATED BY '\t';
```

```
add FILE weekday_mapper.py;
```

```
INSERT OVERWRITE TABLE u_data_new  
SELECT TRANSFORM (userid, movieid, rating, unixtime) USING  
'python weekday_mapper.py'  
AS (userid, movieid, rating, weekday) FROM u_data;
```

d) **(6 pts)** Run the following queries using Hive on the **u_data_new** table and provide a **screenshot** of the results for each one:

SELECT weekday, COUNT(*) FROM u_data_new GROUP BY weekday;

**SELECT weekday, COUNT(*) as Total
FROM u_data_new GROUP BY weekday
ORDER BY Total;**